



The University of Jordan

School of Engineering
Department of Computer Engineering

Decentralized Voting System on Ethereum Blockchain

Supervisor:
Dr. Ramzi Saifan

Author(s):

Mohammad Sohail Saleem 0193391
Saleem

Omar Hussam Aldeen 0202652
Abdallah AlSheikhDurrah

Jun 1st, 2025

ETHICAL STATEMENT

We, the undersigned students, certify and confirm that the work submitted in this project report is entirely our own and has not been copied from any other source. Any material that has been used by other sources has been properly cited and acknowledged in the report.

We are fully aware that any copying or improper citation of references / sources used in this report will be considered plagiarism, which is a clear violation of the Code of Ethics of the University of Jordan.

We further certify and confirm that we had no external help without the approval of our supervisor and proper acknowledgment when it is due. We certify and affirm that we never at any point commissioned a 3rd. party to do the work or any part of it on our behalf regardless of the amount charged or lack thereof. We also acknowledge that if suspected and thereafter proven that we have commissioned a 3rd. party to do any part of this project that we risk failing the entire project.

We certify and confirm that all results presented in this project are true with no manipulation of data or fraud, that any statistics done, or surveys collected are conducted with the highest degree of scientific fidelity and integrity, and that if proven otherwise, we risk failing the entire project. We acknowledge that for any data collected, we have taken all the steps necessary in applying for proper authorizations if deemed necessary, and that all user data collected is subject to the utmost degrees of privacy and anonymity.

Mohammad Sohail Saleem Saleem

Jun 1st, 2025

Signature:

Omar Hussam Aldeen abdallah

AlSheikhDurrah

Jun 1st, 2025

Signature:

SUPERVISOR CERTIFICATION

I hereby certify that the students in this project have **successfully finished** their senior year project and by submitting this report they have fulfilled in partial the requirements of B.Sc. Degree in Computer Engineering.

☐

I hereby certify that the students in this project have not completed their senior year graduation project and **I do not approve** that they proceed to the discussion.

☐

I suspect that the students have **violated** one or more of the clauses in the **ethical statement** and I suggest that an investigation committee look into the matter.

☐

Dr. Ramzi Saifan

Signature:

DEDICATION

We dedicate this project to our families, friends, and mentors who have supported us throughout this journey. Their encouragement, patience, and guidance have been invaluable in helping us reach this milestone.

We would also like to express our gratitude to our academic advisors and instructors, whose knowledge and expertise have inspired us to pursue innovation and excellence in our work.

SYMBOLS, ABBREVIATIONS, AND ACRONYMS

Acronym	Definition
ABET	Accreditation Board for Engineering and Technology
ABI	Application Binary Interface
API	Application Programming Interface
CSS	Cascading Style Sheets
DB	Database
DApp	Decentralized Application
ETH	Ethereum (Cryptocurrency)
EVM	Ethereum Virtual Machine
GAN	Generative Adversarial Network
HTML	Hypertext Markup Language
GUI	Graphical User Interface
IPFS	InterPlanetary File System
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
MFA	Multi-Factor Authentication
PoS	Proof of Stake
PoW	Proof of Work
RPC	Remote Procedure Call
SC	Smart Contract
SQL	Structured Query Language
UI	User Interface
UX	User Experience
VM	Virtual Machine
VS Code	Visual Studio Code (IDE used in development)
ZKP	Zero-Knowledge Proof
Truffle	Ethereum Development Framework
MetaMask	Browser Wallet for Ethereum Blockchain Interaction
Web3.js	JavaScript Library for Blockchain Interaction

ABSTRACT

Traditional voting systems often suffer from security vulnerabilities, inefficiencies, and a lack of transparency. This project introduces a decentralized voting system built on the Ethereum blockchain to address these issues and improve trust in electoral processes. By leveraging smart contracts, the system automates vote casting and counting, ensuring that votes are immutable, verifiable, and publicly auditable.

The web-based interface allows users to log in, authenticate via MetaMask, and cast votes securely. All voting transactions are recorded on the blockchain, with real-time visualization provided through Chart.js. The system was deployed and tested within the University of Jordan's environment using a local Ethereum setup (Ganache), and results demonstrated high security, accurate vote tracking, and successful enforcement of voter uniqueness.

While this implementation was tailored to university elections at the University of Jordan, the system itself is highly adaptable. By modifying configuration parameters such as user roles, bloc categories, and institutional identifiers, the platform can be repurposed for civic elections, corporate voting, or other organizational use cases.

Future enhancements may include optimizing gas usage, integrate mobile platforms, and implement privacy-preserving technologies such as zero-knowledge proofs to further improve confidentiality and scalability.

Table of Contents

ETHICAL STATEMENT.....	III
SUPERVISOR CERTIFICATION	V
DEDICATION.....	VII
SYMBOLS, ABBREVIATIONS, AND ACRONYMS	VIII
ABSTRACT.....	IX
LIST OF FIGURES	XIII
LIST OF TABLES.....	XIV
CHAPTER 1 INTRODUCTION	1
1.1 Problem Definition.....	1
1.2 Proposed Solution	1
1.3 Project Deliverables	2
1.4 Project Impact	2
CHAPTER 2 RELATED WORK.....	3
2.1 Decentralized Voting System Using Ethereum Blockchain	3
2.2 Blockchain-Based E-Voting Framework	3
2.3 DVTChain: A Secure Blockchain Voting Mechanism	4
2.4 Smart Contracts and Election Security	4
2.5 Summary Table - Comparison of Blockchain Voting Solutions	5
Chapter 3: Blockchain Fundamentals	6
3.1 Introduction to Blockchain	6
3.2 Advantages of Blockchain Technology.....	7
3.3 Working Principle of Blockchain	7
3.4 Preventing Double Voting and Ensuring Data Integrity	8
3.5 Forks and the Longest Chain Rule	8
3.6 The Future of Blockchain in Voting.....	9
CHAPTER 4 SOLUTION DESCRIPTION AND IMPLEMENTATION	10
4.1 Introduction	10
4.2 Technologies Used	10
Code Structure	11
4.3 System Architecture	13
Blockchain-Based Voting System.....	14
4.4 System Flowchart and Logic.....	14

4.5 Code Implementation	18
CHAPTER 5 SYSTEM DEPLOYMENT AND DEMO INSTRUCTIONS	26
5.1 Introduction	26
5.2 Software and Tools Required	26
5.3 Setting Up Ganache.....	27
5.4 Setting Up MetaMask.....	28
5.5 MySQL Database Setup	29
5.6 Backend Setup (Node.js).....	30
5.7 Frontend Setup	30
5.8 Compile and Deploy Smart Contracts.....	31
5.9 Copy ABI Folder to Frontend	31
5.10 Run Backend and Frontend Servers	31
5.11 Testing the System	32
CHAPTER 6 RESULTS AND DISCUSSION	33
6.1 Introduction	33
6.2 System Testing Overview	33
6.2.1 Roles Tested	33
6.2.2 Bloc Setup.....	33
6.3 Screenshots and Visual Evidence.....	34
6.4 Discussion	38
6.4.1 Achievements	38
6.4.2 Challenges and Solutions.....	38
6.4.3 Lessons Learned	39
6.4.4 Comparison to Traditional Systems	39
6.5 Summary	40
CHAPTER 7 CONCLUSIONS AND FUTURE WORK.....	41
7.1 Conclusions	41
7.2 Limitations	41
7.3 Future Work	41
7.4 Final Remarks	42
REFERENCES	43
APPENDICES	44

LIST OF FIGURES

FIGURE 1: STRUCTURE OF A BLOCKCHAIN [8]	6
FIGURE 2: SYSTEM ARCHITECTURE DIAGRAM SHOWING INTERACTION BETWEEN FRONTEND, BACKEND, BLOCKCHAIN, METAMASK, AND DATABASE	14
FIGURE 3: USER AND ADMIN VOTING FLOWCHART	16
FIGURE 4: FRONTEND CODE CALLING THE SMART CONTRACT FUNCTIONS VOTE() AND VOTECOLLEGELOC() TO SUBMIT THE USER'S SELECTIONS.	19
FIGURE 5: FRONTEND FUNCTIONS CALLING GETUNIVERSITYLOC() AND GETCOLLEGELOC() TO RETRIEVE AND DISPLAY LIVE VOTE COUNTS FROM THE BLOCKCHAIN.	20
FIGURE 6: IMPLEMENTATION OF THE LOGIN ENDPOINT (POST /LOGIN). THE BACKEND AUTHENTICATES THE USER, ASSIGNS A ROLE, AND RETURNS A SIGNED JWT TOKEN WITH ROLE AND COLLEGE INFORMATION.	22
FIGURE 7: SMART CONTRACT FUNCTIONS TO REGISTER UNIVERSITY AND COLLEGE BLOCs ON THE BLOCKCHAIN. ONCE ADDED, THESE BLOCs ARE IMMUTABLE AND CAN BE REFERENCED DURING VOTE CASTING.	24
FIGURE 8: SMART CONTRACT FUNCTION FOR RECORDING A UNIVERSITY VOTE. THE VOTE HASH IS STORED, THE USER IS MARKED AS HAVING VOTED, AND THE CORRESPONDING BLOC'S VOTE COUNT IS INCREASED. THE HIGHLIGHTED CODE SEGMENT ENSURES:	25
FIGURE 9: GANACHE WORKSPACE SERVER CONFIGURATION SHOWING THE CUSTOM HOSTNAME, PORT, AND NETWORK ID SETTINGS USED FOR THE LOCAL ETHEREUM BLOCKCHAIN.	28
FIGURE 10: CUSTOM RPC CONFIGURATION	29
FIGURE 11: GANACHE LOCAL BLOCKCHAIN ENVIRONMENT – ACCOUNT OVERVIEW.	32
FIGURE 12: USER LOGIN INTERFACE	34
FIGURE 13: VOTING INTERFACE	35
FIGURE 14: ADMIN DASHBOARD	35
FIGURE 15: METAMASK TRANSACTION CONFIRMATION	36
FIGURE 16: GANACHE SUCCESSFUL TRANSACTION LOG	37
FIGURE 17: GANACHE FAILED TRANSACTION LOG	37
FIGURE 18: COLLEGE BLOC SQL TABLE	38

LIST OF TABLES

TABLE 1 COMPARISON OF BLOCKCHAIN VOTING SOLUTIONS	5
TABLE 2 TECHNOLOGY USED	11

CHAPTER 1

INTRODUCTION

Voting is a fundamental pillar of democratic societies, yet traditional voting systems are often plagued by security vulnerabilities, inefficiencies, and a lack of transparency. Issues such as ballot tampering, voter fraud, and miscounts have been reported in various elections worldwide, undermining public trust in the process. For example, manual vote counting can lead to human errors, while electronic voting machines are susceptible to hacking or system failures. Ensuring the integrity of elections through secure and verifiable voting mechanisms is essential to maintaining public confidence and upholding democratic fairness.

This project aims to address these challenges by leveraging blockchain technology to develop a secure and transparent voting system. Blockchain provides a decentralized and tamper-resistant platform that enhances vote security and accuracy. By using smart contracts, the system automates vote verification and counting, reducing human error and the risk of manipulation.

This chapter presents the problem statement, the proposed solution, expected project deliverables, and the overall impact of the system.

1.1 Problem Definition

Traditional voting systems face multiple challenges that can undermine the integrity of elections. Fraudulent practices such as ballot stuffing, voter impersonation, and vote tampering have been widely reported in elections across different countries. These issues compromise the accuracy of election outcomes and diminish public trust in democratic institutions [2].

Additionally, manual vote counting is prone to human errors, which can lead to miscounts, delays, and disputes over election results. The 2000 U.S. presidential election, for example, highlighted how issues with manual ballot counting can cause major political and legal conflicts. Furthermore, centralized voting systems create a single point of failure, making them susceptible to cyberattacks and data manipulation. In 2016, allegations of electronic voting hacking raised concerns about the security of electronic voting machines in several countries [2].

Another major issue is the lack of transparency and verifiability in traditional voting methods. Once a vote is cast, voters have no way to track whether it was recorded correctly or counted as intended. Electronic voting systems, while improving speed, often operate as black boxes where voters must trust the system without any means of verification. This lack of transparency can lead to distrust, especially in highly contested elections.

1.2 Proposed Solution

This project proposes a blockchain-based voting system to enhance transparency, security, and accuracy. Blockchain technology ensures decentralization, meaning that no single entity can alter votes once they are recorded on the immutable ledger. By utilizing Ethereum smart

contracts, the system automates critical election processes, including voter authentication, vote casting, and vote tallying, reducing the risks associated with human intervention.

Additionally, blockchain technology enhances verifiability and trust by allowing voters to track their votes without compromising anonymity. Using cryptographic techniques, each vote is securely recorded and publicly auditable, ensuring that election outcomes are tamper-proof. Furthermore, smart contracts enforce election rules automatically, preventing unauthorized access or vote duplication without the need for third-party oversight.

1.3 Project Deliverables

- A blockchain-based voting system prototype.
- Smart contracts to manage voter authentication and vote counting.
- A front-end interface for voters to cast their votes securely.
- A backend system to connect the blockchain with the user interface.
- Documentation detailing system architecture, implementation, and security considerations.

1.4 Project Impact

Implementing blockchain in voting systems can enhance trust in elections by eliminating fraudulent activities and ensuring transparency. The system provides secure and verifiable elections, reducing reliance on central authorities. Additionally, it streamlines the voting process, making it more efficient and accessible to a wider audience, including remote voters[3].

Although the current implementation was designed for the University of Jordan, the architecture is modular. With minor adjustments—such as modifying bloc types, role definitions, or organizational labels—it can be deployed in a variety of other contexts, including universities, companies, NGOs, and public-sector elections.

CHAPTER 2

RELATED WORK

Introduction

Blockchain-based electronic voting (e-voting) systems have gained significant attention due to their ability to enhance transparency, security, and trust in elections. Several research works and implementations have explored different approaches to decentralized voting mechanisms. This section reviews relevant works that relate to our project.

2.1 Decentralized Voting System Using Ethereum Blockchain

One prominent decentralized e-voting system leverages the Ethereum blockchain to conduct transparent elections by removing intermediaries. In this model, Ethereum smart contracts are used to automate the processes of vote collection and verification. Each voter receives a unique digital identity, and once a vote is cast, it is stored immutably on the blockchain, rendering it tamper-proof. Additionally, the system facilitates remote participation through a web-based interface [1].

The system has proven to be effective in preventing double voting and vote tampering. It also supports real-time tracking of election outcomes, thereby increasing auditability. However, two major weaknesses are evident. First, high Ethereum gas fees can significantly elevate election costs. Second, the scalability of the system is constrained by Ethereum's limited transaction throughput. Compared to this solution, our work introduces optimizations to reduce gas fees during smart contract execution. Furthermore, our implementation strengthens identity verification to further reduce the risk of voter fraud [1].

2.2 Blockchain-Based E-Voting Framework

A separate study by Missa Lamsani et al. proposes a blockchain-based e-voting framework that also uses Ethereum to automate election tasks. This system integrates Solidity-based smart contracts for voter registration, authentication, and vote casting. It relies on the Ethereum Virtual Machine (EVM) to ensure vote transparency and validates vote integrity through cryptographic hashing [2].

The results suggest that the framework significantly reduces the risk of vote tampering when compared to traditional electronic voting systems. Moreover, the use of smart contracts for voter registration prevents unauthorized modifications. Despite its strengths, the system exhibits notable weaknesses. Delays in Ethereum's transaction confirmations can hinder performance, and the absence of a robust identity verification process opens the door to potential abuse by malicious users.

In contrast, our system enhances scalability by incorporating Layer 2 solutions such as zk-Rollups. We also integrate zero-knowledge proofs to bolster voter privacy while preserving transparency [2].

2.3 DVTChain: A Secure Blockchain Voting Mechanism

DVTChain is another notable contribution proposed by researchers from King Saud University. This system emphasizes voter anonymity, security, and fairness. Built on Ethereum 2.0, DVTChain automates election processes using smart contracts and implements anonymous voter registration to enhance privacy [3].

The system uses public-key cryptography to encrypt votes and delays decryption until the election concludes. Moreover, by leveraging Ethereum 2.0's Proof-of-Stake consensus, the framework achieves lower gas fees and improved performance [3].

Compared to DVTChain, our project focuses more on comprehensive identity management and transparent result visualization while still maintaining privacy through cryptographic mechanisms [3].

2.4 Smart Contracts and Election Security

Several studies have also explored the use of smart contracts to enhance election security in blockchain-based systems. Shahzad et al. (2019) proposed a novel approach involving a proof-of-completeness algorithm designed to ensure the integrity of votes. In their model, votes are securely recorded inside immutable blockchain blocs, thereby preventing tampering or data loss. This model stands out for its rigorous focus on safeguarding election data against unauthorized changes [4].

In a different context, Dagher et al. (2018) introduced BroncoVote, a smart contract-based voting solution developed for use in university elections. BroncoVote aims to decentralize the voting process and improve transparency by storing votes across distributed ledgers. Like Shahzad's model, it ensures that votes are not subject to tampering once recorded[4].

While both studies contribute meaningfully to secure e-voting, they also reveal certain limitations. The proof-of-completeness algorithm, though robust, requires significant computational resources, which could make it costly to implement on a large scale. BroncoVote, on the other hand, raises privacy concerns, particularly regarding voter identity exposure due to insufficient anonymization methods [4].

In contrast, our system leverages zero-knowledge proof to maintain a balance between verifiability and voter anonymity. Additionally, by optimizing the underlying blockchain consensus mechanisms, we reduce computational overhead, making our system more efficient and cost-effective [4].

2.5 Summary Table - Comparison of Blockchain Voting Solutions

Table 1 Comparison of Blockchain Voting Solutions

Study	Blockchain Used	Security Features	Weaknesses	How Our Work Differs
GitHub Project [1]	Ethereum	Smart contract-based vote storage, decentralized auditing	High gas fees, limited scalability	Our system reduces gas usage by optimizing smart contract vote storage and limiting blockchain calls to necessary transactions only.
Lamsani et al. (2020) [2]	Ethereum	Smart contracts for vote validation, cryptographic hashing	Scalability issues, lacks strong identity verification	Our system improves efficiency through backend-managed block data and uses hashed email authentication combined with MetaMask wallet verification.
DVTChain [3]	Ethereum 2.0	Voter anonymity, encryption of votes, PoS consensus	Vulnerable to validator collusion	Our system ensures vote integrity using hashed email-based identity mapping and prevents duplicate voting without relying on validator roles or PoS consensus.
Shahzad et al. (2019) & Dagher et al. (2018) [4]	Ethereum	Proof-of-completeness, decentralized vote storage	High computational costs, privacy concerns	While we do not use zero-knowledge proofs, our system preserves voter privacy by hashing all vote data before submission to the blockchain, ensuring anonymity and auditability.

Chapter 3: Blockchain Fundamentals

3.1 Introduction to Blockchain

Blockchain is a decentralized and distributed digital ledger system that enables the secure recording of data and transactions across a network of computers. Unlike traditional databases controlled by a central authority, a blockchain network is maintained collaboratively by its participants (known as nodes), each of which stores a copy of the entire ledger.

This technology was first introduced in 2008 by an anonymous person or group under the pseudonym Satoshi Nakamoto, primarily to serve as the infrastructure for Bitcoin — the first cryptocurrency. However, since its inception, blockchain has evolved far beyond digital currencies and is now widely explored for applications like supply chain tracking, digital identity management, and most notably, electronic voting.

Each unit in the blockchain is called a block, and it contains:

- A list of transactions or data
- A timestamp
- A cryptographic hash of the previous block
- A unique identifier (hash) for itself

Analogy: Think of blockchain as a transparent and tamper-proof digital notebook that everyone can see, but no one can change without consensus.

As you can see in Figure1 These blocks are chained together in a linear sequence, forming an unbreakable link were altering one block would break the chain unless all subsequent blocks are also altered a practically infeasible task. This cryptographic linking ensures tamper resistance, a key feature of blockchain systems.

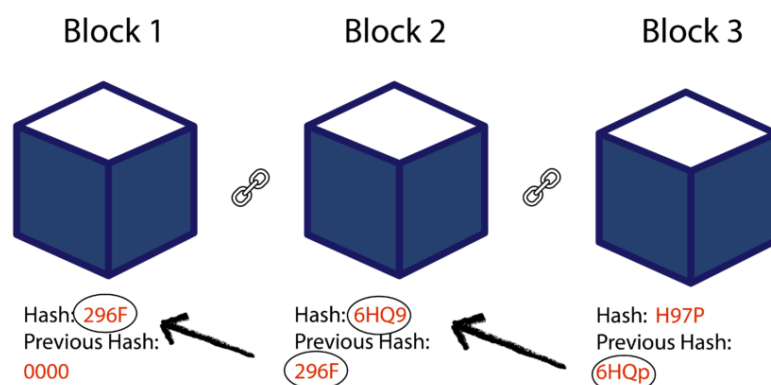


Figure 1: Structure of a Blockchain [8]

3.2 Advantages of Blockchain Technology

Blockchain offers a range of benefits, especially when applied to domains that require trust, transparency, and security. In the context of voting systems, these advantages become even more significant.

- **Decentralization:** Removes the need for a central authority (like an election commission or server), thus eliminating a single point of failure.
- **Transparency:** All transactions (e.g., votes) are publicly verifiable on the blockchain. Anyone can audit the system and verify the integrity of results without exposing individual voter choices.
- **Immutability:** Once recorded, a transaction cannot be changed or deleted. This prevents any retroactive vote manipulation.
- **Security:** Transactions are secured using advanced cryptographic algorithms. Unauthorized access or fraud becomes extremely difficult.
- **Auditability:** Every transaction is time-stamped and traceable, allowing for efficient post-election auditing without revealing voter identities.
- **Cost Efficiency:** Reduces reliance on physical infrastructure, third-party observers, and manual verification — leading to lower administrative costs.

These features collectively build trust in digital elections and promote a more accessible, verifiable, and resilient democratic process.

3.3 Working Principle of Blockchain

Blockchain operates on the principle of distributed consensus — meaning that no single entity controls the network. Instead, all nodes (participants) in the system must agree (reach consensus) on the validity of a new block before it can be added.

When a user casts a vote (or submits a transaction), the following steps occur:

1. **Broadcast:** The transaction is shared across the network.
2. **Validation:** Each node independently verifies it according to predefined rules.
3. **Consensus Mechanism:** The network uses an algorithm to agree on the next block. The most common early method is Proof of Work (PoW).
4. **Block Addition:** Once consensus is reached, the block is added to the chain, and all nodes update their local copy.

Proof of Work (PoW)

In PoW, nodes called miners solve complex mathematical puzzles to validate new blocks. The first one to solve it earns the right to add the block and is rewarded. While secure, PoW is computationally expensive and energy intensive.

In voting systems, this structure prevents fraudulent additions and ensures that only legitimate votes are recorded in the immutable ledger.

3.4 Preventing Double Voting and Ensuring Data Integrity

Blockchain is particularly well-suited to prevent double voting — a major concern in electronic elections. It achieves this through:

- **Cryptographic Hashing:** Every block includes a hash of the previous block, creating a chain of dependency. Any tampering with one block changes its hash, which breaks the chain and alerts the network.
- **Consensus Enforcement:** Modifying historical vote records on a blockchain would necessitate re-mining not only the altered block but also all subsequent blocks. Furthermore, the attacker must achieve consensus by outpacing the cumulative computational work of the honest nodes—a task that is computationally prohibitive and practically infeasible. Even in the hypothetical scenario where such an attack succeeds on a single node, the distributed nature of blockchain ensures that other nodes retain the legitimate version of the ledger. Due to the protocol's reliance on majority agreement, any conflicting version would be automatically rejected by the network, preserving the integrity and consistency of the original data.
- **Unique Voter Identification:** Each vote can be tied to a unique hashed ID of the voter (not the raw email or name), preserving privacy while maintaining vote uniqueness.

Together, these mechanisms form a tamper-proof digital ballot box that ensures each eligible user votes once, and that their vote remains unchanged and traceable throughout the process.

3.5 Forks and the Longest Chain Rule

In decentralized systems, forks are temporary splits in the blockchain when two valid blocks are created simultaneously. This can happen due to:

- Network latency
- Multiple miners solving the PoW at the same time

Each node initially follows the first block it receives. Over time, one branch becomes longer as more blocks are added. Blockchain resolves this using the Longest Chain Rule:

The chain with the most computational work (longest valid chain) is considered the true one. All nodes switch to it, and the shorter branch becomes an orphan chain.

Fork Example:

- Chain A: Block 100 → Block 101A → Block 102
- Chain B: Block 100 → Block 101B

When Block 102 is added to 101A, Chain A becomes the longest. All honest nodes discard Chain B and adopt Chain A.

This mechanism maintains consistency, prevents duplication, and ensures finality in vote records.

3.6 The Future of Blockchain in Voting

While blockchain-based voting systems are still emerging, they show immense promise:

- **Wider Government Adoption:** Estonia, Switzerland, and Brazil have already explored blockchain for public sector transparency.
- **Mobile and Remote Voting:** Blockchain combined with secure apps enables remote voting, increasing accessibility for citizens abroad or with mobility challenges.
- **Biometric Integration:** Biometric ID (e.g., fingerprint, iris) can be hashed and linked to a vote without revealing personal details — enhancing security without compromising anonymity.
- **Interoperability Standards:** Ongoing efforts aim to create universal voting protocols that work across different blockchain platforms.
- **Greener Consensus Mechanisms:** Alternatives to PoW — such as Proof of Stake (PoS) and Directed Acyclic Graphs (DAGs) — provide the same security guarantees with lower environmental impact.

With ongoing research and pilot programs, blockchain voting could become a cornerstone of next-generation digital democracies.

CHAPTER 4

SOLUTION DESCRIPTION AND IMPLEMENTATION

4.1 Introduction

This chapter presents a comprehensive technical explanation of the design and implementation of our blockchain-based voting system. The solution leverages decentralized technologies to address the common vulnerabilities of traditional electronic voting systems, such as tampering, vote duplication, and lack of transparency.

Our goal was to design a system that enables:

- Fair and transparent university elections.
- Separation of roles (admin vs. user).
- Double-vote prevention through blockchain identity mapping.
- Real-time results and voting chart updates.

4.2 Technologies Used

To build our blockchain-based voting system, we carefully selected a set of tools and technologies that are not only widely adopted in the industry but also offer ease of use, strong documentation, and active community support. These technologies span multiple layers of our system—ranging from smart contract development to backend infrastructure and frontend interface design—allowing for smooth integration and reliable performance across components.

The smart contract layer is implemented using **Solidity**, the primary programming language for Ethereum. It enables us to define core functionalities such as vote casting, identity verification, and access control directly on the blockchain, ensuring secure and tamper-proof logic execution.

For development and testing, we used **Ganache**, a local Ethereum blockchain simulator. Ganache allows developers to deploy and test smart contracts in a controlled environment without incurring real-world costs or delays, enhancing our development workflow.

Truffle complements this setup as our smart contract development framework. It handles contract compilation, migration, and automated testing, helping us streamline deployment and maintain code quality throughout the development process.

Instead of relying on backend communication to store or retrieve voting results, our system enables users to interact directly with the blockchain through MetaMask, a widely used browser-based cryptocurrency wallet. MetaMask serves as a secure bridge between the user and the blockchain by handling authentication and allowing users to digitally sign transactions. This integration ensures that all voting actions—such as casting a vote—are authorized, transparent, and tamper-proof, thereby enhancing both the security and trustworthiness of the system.

On the server side, **Node.js** serves as our runtime environment. Its asynchronous, event-driven nature makes it well-suited for handling blockchain requests and user sessions efficiently. Built on top of Node.js, **Express.js** provides the backend framework for developing APIs that connect the frontend with smart contracts and the database.

Configuration management is handled by **Dotenv**, which allows us to securely store sensitive environment variables such as private keys and database credentials in external configuration files. This promotes security and flexibility across deployment environments.

We use **MySQL** as our relational database to store persistent off-chain data such as user information, voting records, and bloc metadata. It provides efficient querying and robust support for complex relationships between entities.

On the Frontend, we designed our interface using **HTML** and **CSS** to structure and style the voting system's pages, including login, voting, and results dashboards. Finally, **JavaScript** is used to manage dynamic client-side logic, enabling real-time updates and communication with both the backend and the blockchain.

Table 2 below summarizes the technologies and their respective roles within our project.

Table 2 technology used

Technology	Purpose
Solidity	Smart contract programming language
Ganache	Private Ethereum blockchain simulator
Truffle	Smart contract compiler, deployer
MetaMask	Wallet extension for secure blockchain interaction
Node.js	Runtime environment
Express.js	Backend framework
Dotenv	Securely stores environment variables
MySQL	Stores users, blocs, and relationships
HTML+CSS	Frontend style
JS	Frontend

Code Structure

The project is organized into four main layers: frontend, backend, blockchain smart contracts, and database. This modular architecture ensures separation of concerns, reusability, and easier debugging.

- **Backend:**

The backend is responsible for server-side logic and integration with the MySQL database and Ethereum blockchain. It uses Express.js for API routes and handles authentication, user management, vote submissions, and results aggregation. Key files include:

1. backendwauto.js: Main server file connecting database, smart contract, and routes.
2. env: Stores sensitive config like DB credentials and ports.
3. sqldata.sql: SQL schema for tables (users, votes, blocs).
4. hashPasswords.js: Utility for securely hashing passwords using bcrypt.

- **Blockchain:**

The blockchain folder contains the Solidity smart contract 'VotingSystem.sol' which defines voting logic, access control, and vote integrity. It also includes the build output (JSON ABI) and migration scripts used by Truffle. Components include:

1. contracts/VotingSystem.sol: Core smart contract.
2. build/contracts/VotingSystem.json: Compiled ABI + network metadata.
3. migrations/1_initial_migration.js: Deployment script for Truffle.

- **Frontend:**

The frontend provides the user interface for voters and admins. It is written in HTML, CSS, and JavaScript, and interacts with both the backend and the smart contract using Ethers.js. Key elements include:

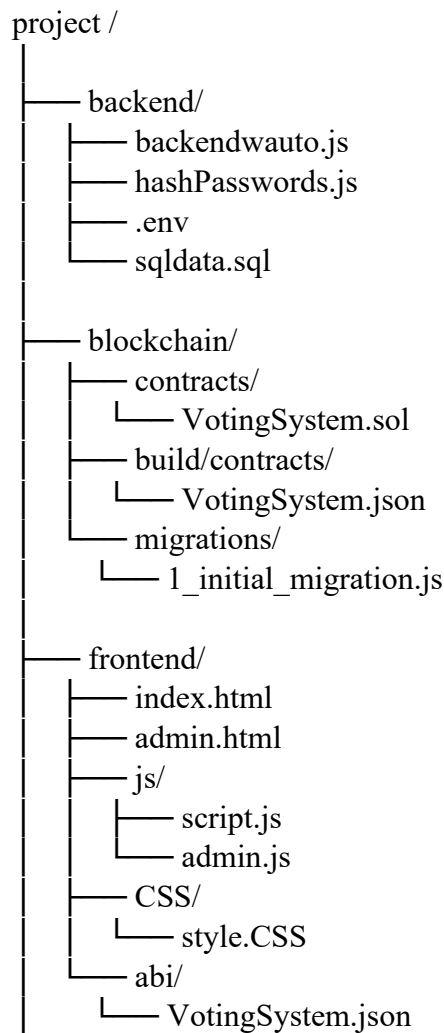
1. index.html: Voter interface (login, voting, results).
2. admin.html: Admin dashboard for managing blocs.
3. js/script.js: Logic for voting, charts, login, and MetaMask.
4. js/admin.js: Admin-specific JavaScript logic.
5. CSS/style.CSS: All UI styles, themes, and responsive design.
6. abi/VotingSystem.json: ABI used by frontend to connect to the smart contract.

- **Database (MySQL):**

The MySQL database manages persistent data such as users, university blocs, college blocs, and votes. It ensures data normalization, indexing, and efficient querying for real-time results. It interacts with both backend APIs and the blockchain for synchronization.

This clear separation of layers enhances modularity, allows for independent development and testing, and makes the system scalable and secure for deployment in real-world decentralized voting scenarios.

The simplified directory tree structure is as follows:



4.3 System Architecture

Traditional Voting System (Backend-Centric)

In traditional voting systems, the **backend** plays a central managerial role. The **front end** communicates with the backend, which then interacts with a **MySQL database** to cast and retrieve votes. While this setup is functional, it introduces several security vulnerabilities:

- **Database Vulnerability:** If the database is not properly secured, it becomes a target for attackers. Even with security measures, experienced attackers may breach the system and manipulate vote records.
- **Undetectable Attacks:** In some cases, these breaches may go undetected, undermining the integrity of the entire election.
- **Man-in-the-Middle Attacks (MITMAs):** Attackers can intercept and alter the communication between the frontend and backend, modifying votes before they are recorded.

Blockchain-Based Voting System

In contrast, the blockchain-based system significantly reduces the backend's responsibilities. To enhance **security** and **prevent MITMAs**, users (via the frontend) directly interact with the **blockchain** through **MetaMask**, a digital wallet.

- **Direct Blockchain Interaction:** Users submit their votes directly to the blockchain without intermediary handling by the backend.
- **Immutable Records:** Once a vote is recorded on the blockchain, it becomes **immutable**, ensuring that it cannot be altered or deleted.
- **Decentralized Trust:** This architecture eliminates single points of failure and central authority manipulation, which is a key strength of blockchain technology.

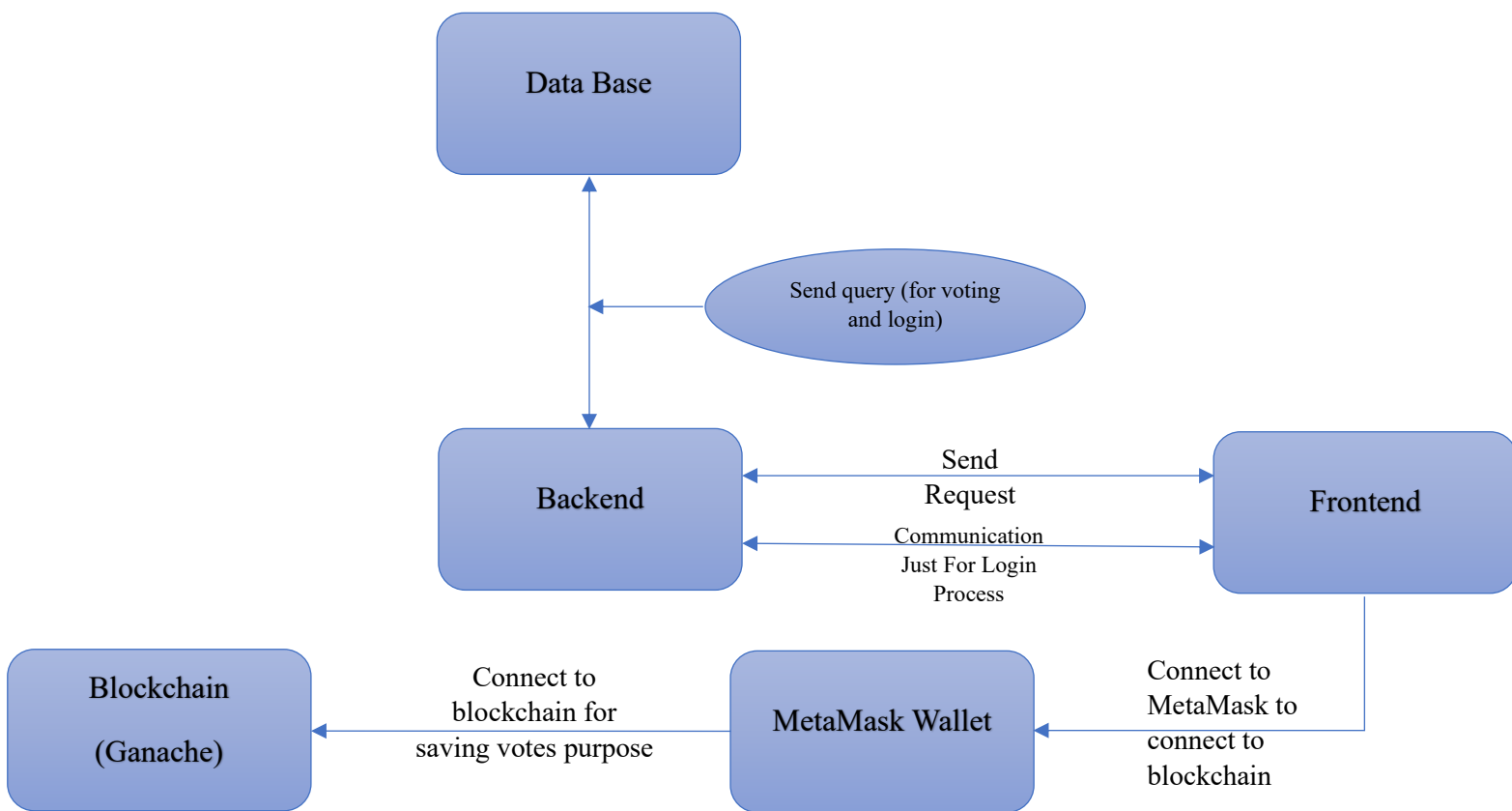


Figure 2: System Architecture Diagram Showing Interaction Between Frontend, Backend, Blockchain, MetaMask, and Database

4.4 System Flowchart and Logic

The system workflow is structured into a series of decision points to ensure secure and role-based access to the voting platform. The flowchart outlines the logical progression of events

for both regular users (students) and administrators (admins), covering login, authentication, voting eligibility, and vote submission.

- The process begins when a user visits the website. The system checks whether the visitor is a user or an admin.
- If the visitor is an admin (identified by the email `admin@voting.com`), they are granted access to the admin dashboard, where they can:
 1. Add new college blocs
 2. Edit university or college bloc names
 3. View vote charts by college or university
- If the visitor is a regular user (student), they are directed to the login page.
- Upon successful login, the system retrieves the user's associated college and role and stores them in the browser's local Storage for later use.
- The user selects a university bloc and a college bloc relevant to their college; both dynamically fetched from the backend.
- The system checks whether the user has already voted by querying the smart contract using their MetaMask wallet address.
- If the user has not voted, the frontend proceeds to submit:
 1. A university bloc vote via the `vote()` function
 2. A college bloc vote via the `voteCollegeBloc()` function
- Before submission, the system verifies:
 1. MetaMask is installed and unlocked
 2. The user has sufficient gas for the transaction
- Upon successful voting, local Storage is updated to track the user's vote and prevent duplicate submissions from the same wallet.

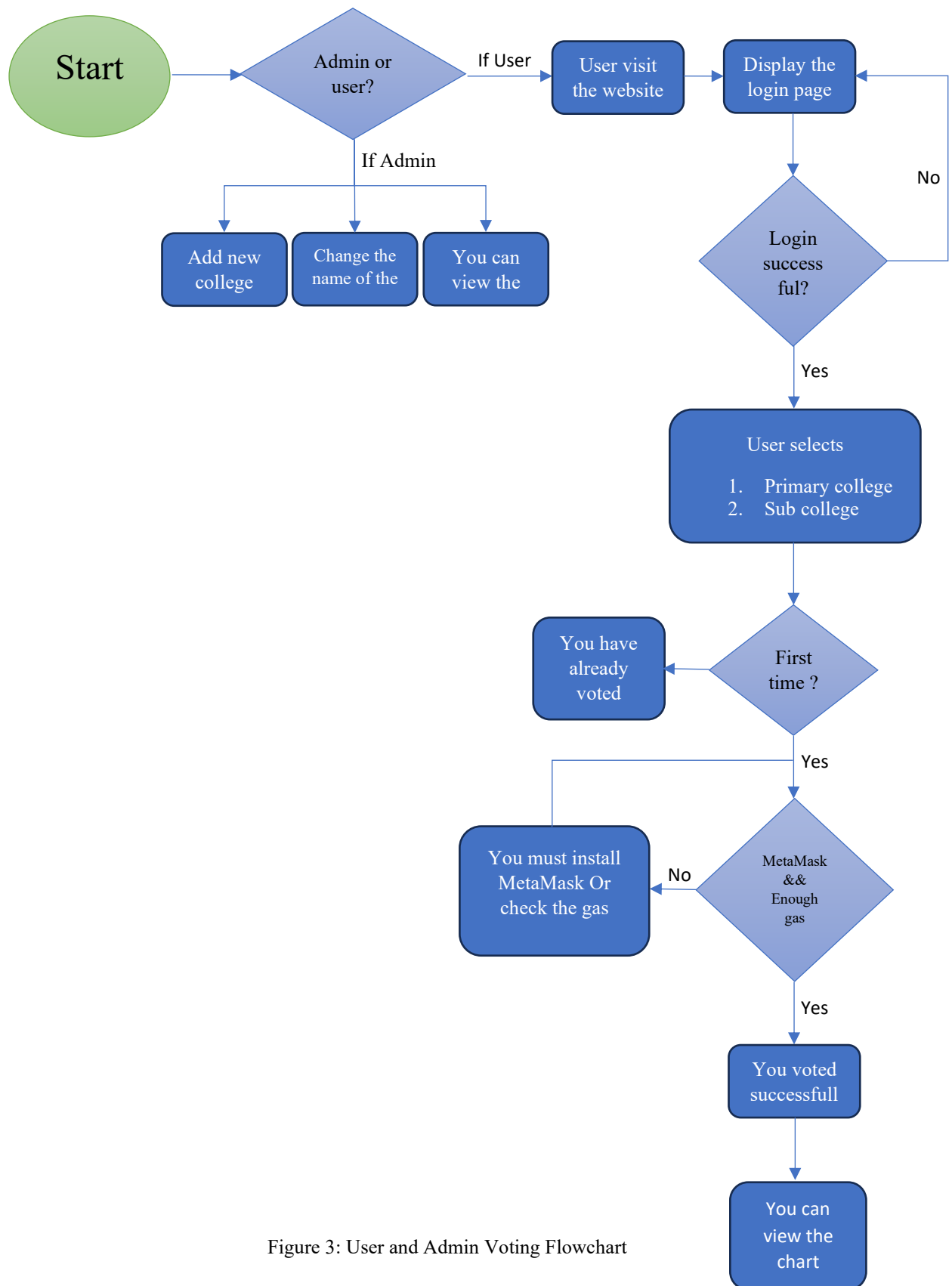


Figure 3: User and Admin Voting Flowchart

Workflow Breakdown by Stages

Stage 1: Login and Role Detection

- Users access the main page and enter their credentials.
- The backend securely verifies credentials using `bcrypt.compare()`, comparing the plaintext input with the hashed password in MySQL.
- Each user is associated with a role (admin or user) and a college.
- If the role is admin, the user is redirected to the admin dashboard; otherwise, they proceed to the voting page.

Stage 2: Bloc Rendering

Upon successful login:

- University blocs are loaded from the database.
- College blocs are filtered based on the user's associated college (stored in local Storage). This ensures that students only vote within their designated college.
- The frontend sends a request to `/college-blocs/:college`, and the response is rendered dynamically as radio buttons or card-style options.

Stage 3: Blockchain Readiness Check

Before any vote is allowed:

- The system checks if MetaMask is installed and connected.
- It verifies if the Ethereum address is already associated with a vote using both local Storage and the smart contract's `hasVoted` mapping.
- It ensures the account has sufficient gas to complete the transaction.

Stage 4: Voting and Hashing

Once validated:

- The frontend generates a vote hash by combining the user's email with the selected bloc ID.
- Two transactions are sent:
 1. `vote(emailHash, voteHash, candidateId)` for university vote
 2. `voteCollegeBloc(voteHash, candidateId)` for college vote
- MetaMask prompts the user to sign and authorize each transaction.

- Upon successful mining:
 1. The smart contract records the vote and updates the corresponding bloc's vote count.
 2. A confirmation message is shown.

Stage 5: Chart Update

- Vote results are fetched in real time from the blockchain using Ethers.js.
- Each bloc's vote count is retrieved via `getUniversityBloc()` or `getCollegeBloc()` methods.
- The data is displayed using Chart.js in dynamic bar charts.

Error Handling and Security Measures

To maintain system robustness:

- **Frontend Validation:** Ensures both university and college blocs are selected before submission.
- **Backend Role Enforcement:** Blocks unauthorized access to admin pages or APIs.
- **Blockchain Vote Locking:** Prevents double voting through `hasVoted` mappings per wallet address.
- **MetaMask Environment Checks:** Ensures users are connected to the correct network (e.g., Ganache) and have sufficient gas.

4.5 Code Implementation

1. Frontend implementation (Client Layer)

The front end of the system is developed using HTML, CSS, and JavaScript. It provides the main interface through which users interact with the voting system. It plays a central role in guiding users through login, vote submission, and viewing results. The frontend also establishes secure communication with the Ethereum blockchain via MetaMask and Ethers.js.

Functional Responsibilities

- Provides the user interface for login, voting, and real-time results display.
- Dynamically adjusts based on the authenticated user's role (admin or student) and associated college.

- Integrates with MetaMask to authenticate users and enable blockchain transaction signing.
- Prevents unauthorized access and repeat voting attempts through conditional rendering logic and state tracking.

Key Tasks Performed

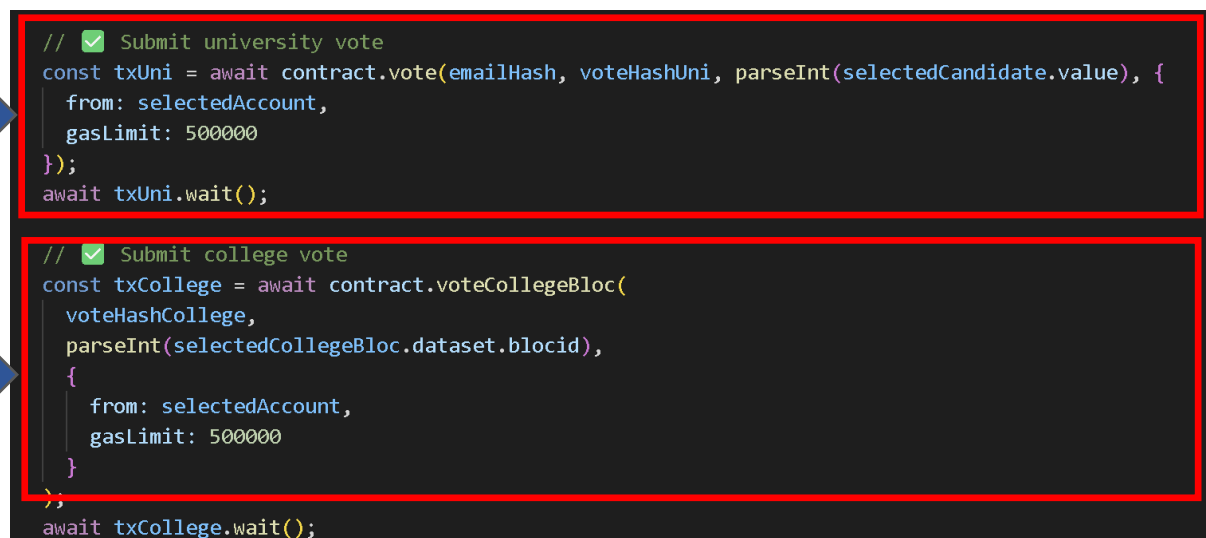
- **UI rendering based on user role and login status.**
- **Submitting votes to the blockchain smart contract.**

When a user submits their selections, the frontend calls two smart contract functions:

1. `vote()` to record the university bloc vote.
2. `voteCollegeBloc()` to record the college bloc vote.

These functions are called with hashed vote data, the selected candidate or bloc ID, and the wallet address. MetaMask is used to authorize and sign transactions. The submitted votes are then recorded immutably on the blockchain.

Figure 4 illustrates the frontend code responsible for submitting the university and college votes.



```
// ✅ Submit university vote
const txUni = await contract.vote(emailHash, voteHashUni, parseInt(selectedCandidate.value), {
  from: selectedAccount,
  gasLimit: 500000
});
await txUni.wait();

// ✅ Submit college vote
const txCollege = await contract.voteCollegeBloc(
  voteHashCollege,
  parseInt(selectedCollegeBloc.dataset.blocid),
  {
    from: selectedAccount,
    gasLimit: 500000
  }
);
await txCollege.wait();
```

Figure 4: Frontend code calling the smart contract functions `vote()` and `voteCollegeBloc()` to submit the user's selections.

The red highlighted areas in the code indicate the specific blockchain function calls that initiate the vote transactions. These blocks also define the sender's account and gas limit, which are essential parameters for execution on the Ethereum network.

Retrieving vote results from the blockchain.

The frontend is also responsible for fetching and displaying up-to-date vote counts for both university and college blocs. It does this by calling:

- `getUniversityBloc(candidateId)`
- `getCollegeBloc(candidateId)`

These asynchronous functions retrieve vote data from the blockchain and parse the results, which are then visualized using Chart.js.

Figure 5 presents the relevant frontend logic for retrieving vote counts.



```
async function getUniversityBloccount(candidateId) {
  try {
    // Call the contract function
    await new Promise(r => setTimeout(r, 2000));
    const result = await contract.getUniversityBloc(candidateId)
    ;
    // result[0] => candidate.id (uint8)
    // result[1] => candidate.voteCount (uint256)
    console.log(`👤 Candidate ID: ${result[0].toString()}`);
    console.log(`🗳️ Vote Count: ${result[1].toString()}`);
  } catch (error) {
    console.error("❌ Error fetching candidate:", error);
  }
}

async function getCollegeBloccount(candidateId) {
  try {
    // Call the contract function
    await new Promise(r => setTimeout(r, 2000));
    const result = await contract.getCollegeBloc(candidateId)
    ;
    // result[0] => candidate.id (uint8)
    // result[1] => candidate.voteCount (uint256)
    console.log(`👤 college ID: ${result[0].toString()}`);
    console.log(`🗳️ Vote Count: ${result[1].toString()}`);
  } catch (error) {
    console.error("❌ Error fetching candidate:", error);
  }
}
```

Figure 5: Frontend functions calling `getUniversityBloc()` and `getCollegeBloc()` to retrieve and display live vote counts from the blockchain.

The highlighted blocks in the figure show where the contract functions are invoked, and how the results are processed for presentation. These results are used to render dynamic bar charts reflecting voting outcomes in real time.

2. Backend implementation (API Layer)

The backend of the system is developed using Node.js and the Express.js framework. It acts as a bridge between the frontend, the database, and the Ethereum smart contract. While the blockchain handles vote immutability and authenticity, the backend performs essential tasks related to authentication, data validation, and synchronization.

Functional Responsibilities

- **Bloc Synchronization:** Ensures that newly added university and college blocs in the database are registered on the blockchain by interacting with the smart contract.
- **API Provisioning:** Offers RESTful endpoints used by the frontend to perform actions such as user login, bloc retrieval, and vote results display.
- **User Authentication:** Verifies user credentials during login using secure hashing techniques (e.g., bcrypt) and returns a JWT token containing the user's role and college.
- **Smart Contract ABI Exposure:** Serves the contract's ABI to the frontend, enabling it to interact with the smart contract via Ethers.js.

Key Backend Endpoints

- **POST /login** – Authenticates user credentials and returns a signed JWT token, including the user's role (admin or user) and associated college.
- **GET /university-blocs** – Retrieves a list of all university blocs from the MySQL database.
- **GET /college-blocs/:college** – Retrieves blocs specific to a given college.

Login Flow Explanation

The POST /login route implements the authentication logic. Upon receiving the user's email and password, the system performs the following steps:

- **Database Lookup:** A SQL query is executed to fetch the user record associated with the provided email.
- **Credential Validation:** The submitted password is compared against the stored password (in practice, this should use bcrypt for hashing).

- **Role Assignment:** If the email matches a predefined administrator account (e.g., admin@voting.com), the role is set to 'admin'; otherwise, it defaults to 'user' or the database-stored value.
- **JWT Token Generation:** A signed JWT token is created, embedding the user's ID and role. This token is sent to the frontend for persistence and authorization.
- **Response:** The backend responds with the login status, token, user role, and associated college.

The highlighted code in Figure 6 shows the exact implementation of this flow.

```
// Login route (POST /login)
// -----
app.post('/login', (req, res) => {
  const { email, password } = req.body;

  console.log("📧 Received Login Request:", { email, password });

  db.query('SELECT * FROM users WHERE email = ?', [email], (err, results) => {
    if (err) {
      console.error('❌ Error fetching user:', err);
      return res.status(500).json({ success: false, message: 'Server error' });
    }

    if (results.length === 0) {
      return res.status(401).json({ success: false, message: 'Invalid credentials' });
    }

    const user = results[0];

    if (password !== user.password) {
      return res.status(401).json({ success: false, message: 'Invalid credentials' });
    }

    // ✅ Step 1: Assign role
    const role = user.email === 'admin@voting.com' ? 'admin' : user.role || 'user';

    // ✅ Step 2: Include role in JWT
    const token = jwt.sign({ userId: user.id, role }, process.env.JWT_SECRET, { expiresIn: '1h' });

    // ✅ Step 3: Send role to frontend
    return res.json({
      success: true,
      message: 'Login successful',
      token: token,
      college: user.college,
      role: role
    });
  });
});
```

Figure 6: Implementation of the login endpoint (POST /login). The backend authenticates the user, assigns a role, and returns a signed JWT token with role and college information.

The green highlighted comments in the code segment outline each step of the process:

- Step 1: Role determination based on email.

- Step 2: JWT token creation using the secret key.
- Step 3: Returning the structured JSON response to the client.

This endpoint is fundamental for determining access control throughout the system and securing user sessions.

3. MySQL Database (Data Layer)

The MySQL database serves as the structured data layer of the voting system. It stores persistent information that is essential for user authentication, bloc management, and vote association. Although the actual vote integrity and counting are managed on-chain via the blockchain, the database remains a vital off-chain component for administrative and analytical tasks.

Key Tables

- **users:** Stores user records, including:
 1. Email
 2. Password (stored as a hashed value using bcrypt)
 3. Role (admin or user)
 4. Associated college
- **university_blocs:** Stores the names and IDs of all university-level blocs (e.g., كتلة اهل الهمّة).
- **college_blocs:** Stores the names and associated college of each college-level bloc (e.g., كتلة احسان, engineering).
- **votes:** Optionally stores the mapping between users and the blocs they voted for (used for additional backend verification and analysis; vote hashes are kept off-chain for privacy and immutability).

Security Practices

To maintain data integrity and ensure reliable operations, the database applies several constraints and safeguards:

- **Foreign Key Constraints:** Enforced between votes and the users / blocs tables to prevent orphan records.
- **Uniqueness Constraints:** Prevents the addition of duplicate bloc names within the same college or university scope.
- **Default Role Enforcement:** New users are automatically assigned the role user unless explicitly elevated to admin.

4. Blockchain implementation (Ethereum via Ganache)

The Ethereum smart contract, VotingSystem.sol, is deployed locally on the Ganache blockchain for testing and development purposes. It is responsible for handling the core voting logic in a decentralized, transparent, and tamper-proof manner.

Key Responsibilities

- **Voter Eligibility**

The smart contract uses the caller's Ethereum wallet address (via msg.sender) to verify voting rights and track whether the user has already submitted a vote. This prevents double voting and enables accountability without revealing identity.

- **Bloc Registration**

New university and college blocs are registered on the blockchain using two dedicated functions:

1. "addUniversityBloc(uint8 blocId)"
2. "addCollegeBloc(uint8 blocId)"

Figure 7 illustrates the code that registers new blocs into their respective arrays on-chain.

```
function addUniversityBloc(uint8 blocId) public {
    universityBlocs.push(UniversityBloc(blocId, 0));
    emit UniversityBlocAdded("University Bloc Added", blocId);
}

function addCollegeBloc(uint8 blocId) public {
    collegeBlocs.push(CollegeBloc(blocId, 0));
    emit CollegeBlocAdded("College Bloc Added", blocId);
}
```

Figure 7: Smart contract functions to register university and college blocs on the blockchain. Once added, these blocs are immutable and can be referenced during vote casting.

- **Vote Submission and Counting**

The contract provides functions that allow users to submit votes for both university and college blocs. It validates that the user (wallet address) has not already voted, stores the vote hash, and increases the vote count for the chosen bloc.

Figure 8 shows the internal logic of the university voting function.



```
// ✓ University Vote: Now also checks emailHash
function vote(bytes32 emailHash, string memory voteHash, uint8 blocId) public {
    require(!voters[msg.sender].hasVotedUniversity, "Already voted for university bloc.");
    require(!emailHasVoted[emailHash], "Email already used to vote.");

    voters[msg.sender].hasVotedUniversity = true;
    voters[msg.sender].universityVoteHash = voteHash;
    voteHashes[voteHash] = true;
    emailHasVoted[emailHash] = true;

    universityBlocs[blocId - 1].voteCount++;

    emit VoteCast(msg.sender, voteHash, "university");
}
```

Figure 8: Smart contract function for recording a university vote. The vote hash is stored, the user is marked as having voted, and the corresponding bloc's vote count is increased. The highlighted code segment ensures:

The highlighted code segment ensures:

1. Each address can vote only once (hasVotedUniversity).
2. Each vote is stored as a hashed string to preserve privacy (voteHash).
3. Duplicate votes are rejected using both email hash and wallet address mappings.

Blockchain-MetaMask Integration

MetaMask is used by the frontend to handle all Ethereum wallet interactions:

- Injects Ethereum accounts into the browser context (window.ethereum).
- Prompts users to approve and sign vote transactions.
- Ensure that the network is connected to the correct environment (Ganache at localhost:7545).

This smart contract design ensures that vote integrity, transparency, and non-repudiation are maintained independently of the backend or database layers.

CHAPTER 5

SYSTEM DEPLOYMENT AND DEMO INSTRUCTIONS

5.1 Introduction

In this chapter, we present a detailed deployment and testing guide for our decentralized voting system. The process includes the installation of development tools, configuration of the blockchain and database, integration of front-end and back-end systems, and deployment of smart contracts.

The main steps are:

- Installing and configuring the necessary software tools.
- Setting up Ganache, a local Ethereum blockchain for development.
- Connecting MetaMask to the blockchain for transaction signing.
- Preparing the MySQL database to manage users and vote data.
- Launching the backend and frontend servers.
- Deploying smart contracts using Truffle.
- Testing the system end-to-end using real blockchain transactions.

Each section begins with an overview and is followed by well-structured instructions to ensure consistent and reliable deployment.

5.2 Software and Tools Required

To implement and test the decentralized voting system, a number of software tools and development platforms were required. This section lists the essential tools used during development, explaining the role of each in the overall system architecture. These tools span backend development, blockchain deployment, frontend testing, and smart contract interaction.

- **Visual Studio Code** – A versatile and lightweight code editor used throughout the project to write, manage, and debug both backend and frontend source code efficiently. It supports extensions that ease smart contract development and frontend testing.
- **Node.js and npm** – Node.js provides the JavaScript runtime needed to run the backend server, while npm (Node Package Manager) is used to install and manage dependencies such as Express, MySQL2, Web3, and other libraries critical for backend development.
- **Ganache** – A local Ethereum blockchain simulation environment, where we deployed and tested smart contracts during development. Ganache allowed us to mimic blockchain behavior in a controlled setting with instant block confirmation and access to multiple test accounts.
- **Truffle Suite** – A powerful development framework for compiling, deploying, and managing smart contracts. We used Truffle to compile our Solidity code and deploy it directly to the Ganache network for testing.

- **MetaMask** – A browser-based cryptocurrency wallet that allowed users to connect to the blockchain securely. It was used to sign and submit transactions, such as casting votes, directly from the frontend interface.
- **MySQL Server** – A relational database management system used to store user credentials, voting bloc information, and vote metadata. The backend server communicates with MySQL to authenticate users and synchronize data with the smart contract.
- **Java Development Kit (JDK) (optional)** – Required by some versions of Truffle or system configurations to support smart contract compilation and execution. It was not always mandatory but useful for compatibility.
- **Live Server extension** – A Visual Studio Code extension that serves frontend HTML and JavaScript files over a custom local IP address. It allowed us to preview the frontend in the browser and simulate the complete voting experience in a local environment.

5.3 Setting Up Ganache

Ganache is a personal Ethereum blockchain used for deploying and testing smart contracts in a local environment. It provides developers with a user-friendly interface and a fully controllable blockchain, making it ideal for simulating real-world blockchain interactions. In our project, Ganache served as the local network on which we deployed smart contracts, tested vote transactions, and monitored gas usage. The steps below outline how we configured Ganache for this purpose.

- **Open Ganache and create a new workspace** – Launch the Ganache application and click on the option to create a new workspace. This workspace will serve as an isolated environment for running our smart contracts during testing.
- **Configure server settings** – Inside the workspace settings, update the following parameters: set the **Hostname** to 0.0.0.0 to allow network-wide access, the **Port** to 7545 (Ganache's default port), the **Network ID** to 1337 for compatibility with Truffle, and make sure **Automine** is enabled to ensure that transactions are confirmed immediately during testing.
- **Save and restart Ganache** – After completing the configuration, save the workspace. Then, restart Ganache to launch the private blockchain instance with your custom settings applied.

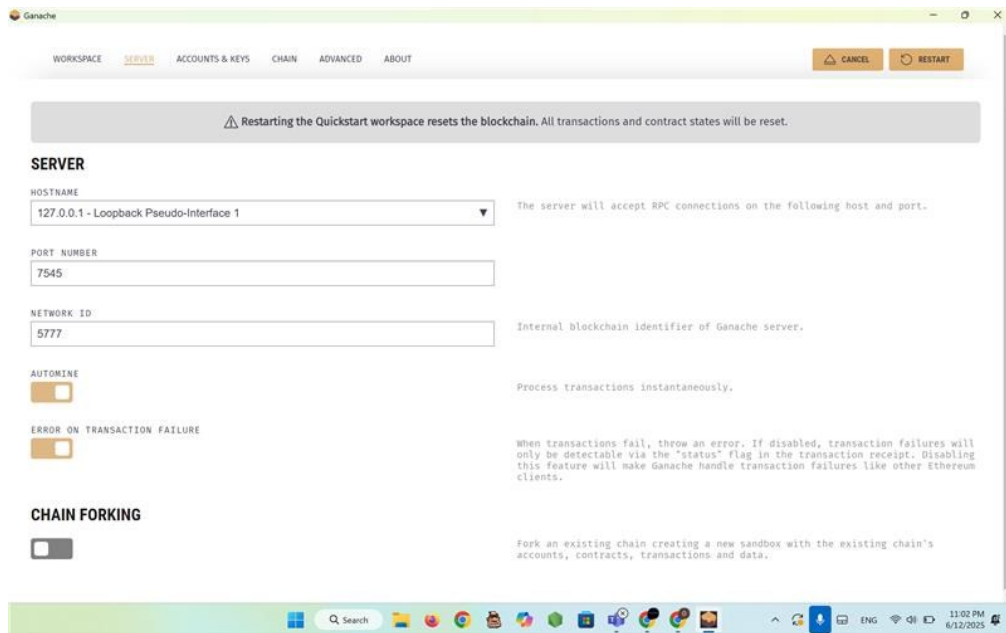


Figure 9: Ganache workspace server configuration showing the custom hostname, port, and network ID settings used for the local Ethereum blockchain.

5.4 Setting Up MetaMask

MetaMask is a browser-based Ethereum wallet that allows users to interact with decentralized applications (DApps) directly from the browser. It is essential for signing blockchain transactions such as casting votes in our system. In our project, MetaMask was configured to connect to the local blockchain running on Ganache. This section describes how to set up MetaMask to communicate with the Ganache network and test accounts.

- **Open MetaMask and add a new network with the provided IP settings** – Access the MetaMask extension in your browser and navigate to the settings to add a new network. Use the same IP address and port as configured in Ganache (e.g., <http://192.168.1.x:7545>). This ensures that MetaMask will point to the local blockchain for all transaction interactions.
- **Import an account using a private key copied from Ganache** – In the Ganache interface, each test account is associated with a private key. Copy one of these keys and import it into MetaMask to gain control over that test account. This imported account will be used to sign transactions on behalf of users during testing.
- **Transfer ETH between Ganache and the MetaMask account** – Ensure your MetaMask account is funded with test ETH by transferring it from another Ganache account if needed. This ETH will be used to cover the gas fees when voting or deploying contracts during testing.
- Figure 14 demonstrates the configuration required to connect MetaMask to Ganache. The custom RPC URL (192.168.0.171:7545), chain ID (1337), and currency symbol (ETH) must match the local blockchain environment. Proper configuration ensures

seamless interaction between the frontend and the blockchain during development and testing.

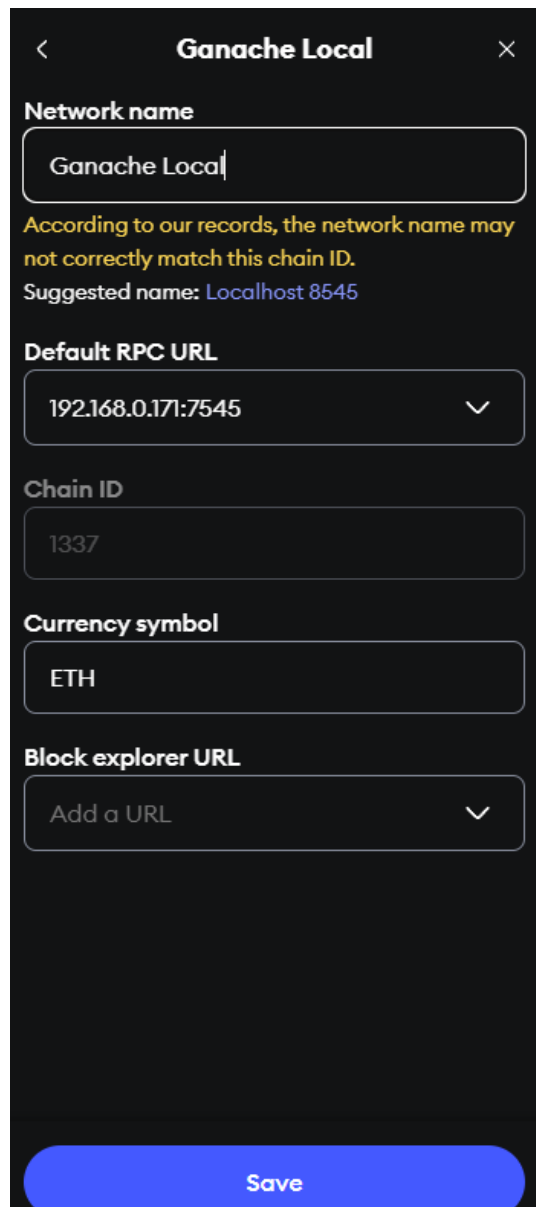


Figure 10: Custom RPC Configuration

5.5 MySQL Database Setup

MySQL serves as the primary data storage system in our voting platform. It is used to store user credentials, bloc information, and vote tracking data. The backend communicates with the database to authenticate users, retrieve voting options, and ensure data consistency. This section outlines how to set up the MySQL environment required for the system.

- **Open MySQL Workbench** – Use either the MySQL Workbench (graphical interface) or the MySQL command-line interface to connect to your local MySQL server and execute SQL commands.

5.6 Backend Setup (Node.js)

The backend is responsible for handling user authentication, communicating with the MySQL database, and interacting with the smart contract on the blockchain. It exposes RESTful APIs for the frontend and ensures secure handling of user credentials and vote-related operations. Node.js is used as the server runtime, and various packages support its functionality. The following steps guide the complete setup of the backend environment.

- **Open VS Code inside the backend project folder** – Launch Visual Studio Code and open the folder containing the backend source files. This directory includes the core server scripts and configuration files.
- **Install backend dependencies using the npm install command** – In the terminal, run the `npm install` command:
`npm install express mysql2 cors dotenv body-parser bcryptjs jsonwebtoken ethers web3`
These libraries are required for HTTP handling, database connection, environment variable management, encryption, JWT authentication, and blockchain interaction.
- **Update a .env file with database and server settings** – Create or edit the .env file to include your database host, user credentials, and the desired backend port. This keeps sensitive configuration details outside the codebase.
- **Run the script hashPasswords.js once to convert any plaintext passwords stored in the database into secure bcrypt hashes** – This script ensures that all user passwords are hashed before login attempts. Execute it with the command `node hashPasswords.js`. It automatically detects any unhashed passwords in the users table and hashes them.
- **Update backendwauto.js to connect to the Ganache network using your IP address** – Modify the Web3 provider section in the backend to match the IP and port used in Ganache (e.g., `http://192.168.x.x:7545`), allowing the server to interact with the local blockchain.
- **Correct the path for loading the VotingSystem ABI** – Make sure the path pointing to the smart contract's ABI file (VotingSystem.json) is accurate, so the backend can correctly read and interface with the deployed contract.

5.7 Frontend Setup

The frontend is the user-facing part of the decentralized voting system. It provides an interface for users to log in, cast their votes, and view real-time results. The frontend interacts with the backend via REST APIs and communicates with the Ethereum blockchain via MetaMask. This section outlines how to prepare the frontend for local deployment.

- **Open VS Code in the frontend folder** – Launch Visual Studio Code and open the folder containing the HTML, CSS, and JavaScript files responsible for the user interface and blockchain interactions.

- **Update BASE_URL in script.js and admin.js** – In the frontend JavaScript files, locate the BASE_URL variable and update it to reflect the backend server's IP (e.g., `http://192.168.x.x:3000`). This allows the frontend to send API requests to the correct backend endpoint.

5.8 Compile and Deploy Smart Contracts

Smart contracts define the rules and logic for voting operations on the blockchain. Before interacting with them, the contracts must be compiled into bytecode and deployed to the local Ethereum network (Ganache). This section outlines how to compile and deploy the voting contract using Truffle, a widely used Ethereum development framework.

- **Navigate to the blockchain folder** – Open a terminal and move into the directory where the smart contract files and Truffle configuration are stored. This is typically named `blockchain` or similar in the project structure.
- **Compile contracts using `truffle compile`** – Run this command to convert the Solidity source code (`.sol` files) into deployable bytecode and ABI files. These files will later be used by both the backend and frontend to interact with the contract.
- **Deploy contracts using `truffle migrate --reset --network development`** – Use this command to deploy the compiled contracts to the local Ganache blockchain. The `--reset` flag ensures that the deployment overwrites any previous version, and `--network development` specifies the target network as defined in the Truffle configuration.

5.9 Copy ABI Folder to Frontend

The ABI (Application Binary Interface) is a JSON representation of the smart contract's structure, including its functions and events. It allows the frontend to interact with the deployed smart contract by knowing how to encode and decode transactions. This section explains how to transfer the ABI to the frontend so it can communicate with the blockchain properly.

- **Copy the `blockchain/build/contracts/VotingSystem.json` file** – After compiling the smart contract, Truffle generates this file. It contains the ABI and bytecode needed by the frontend to interface with the contract on the blockchain.
- **Paste it into the `frontend/js/abi/` directory** – Move the copied file into the designated ABI folder within the frontend project. This allows JavaScript files such as `script.js` and `admin.js` to load the ABI and interact with the smart contract's functions (e.g., `vote()`, `getCandidates()`).

5.10 Run Backend and Frontend Servers

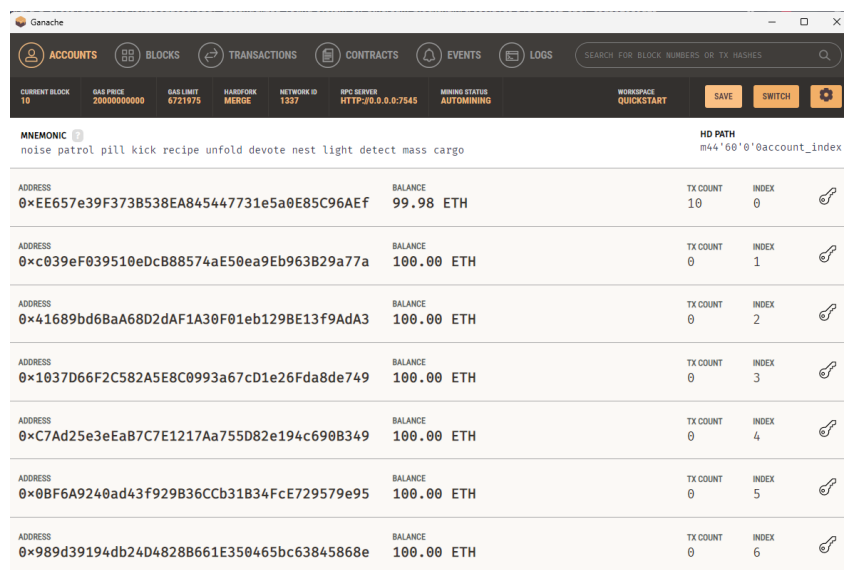
Once the backend, frontend, and database configurations are complete, the final step is to launch the system components to enable user interaction and smart contract execution. This section outlines how to start the backend server, execute the password hashing script, and serve the frontend interface locally.

- **Start the backend using npx nodemon backendwauto.js** – Use this command inside the backend project folder to start the API server. Nodemon will automatically reload the server if any code changes are detected, making development easier.
- **Start the hash script using npx nodemon hashPasswords.js** – Run this script to ensure that all plaintext user passwords stored in the database are converted to secure bcrypt hashes. This script should be executed at least once after inserting new users manually into the users table.
- **Start the frontend using live-server --host=0.0.0.0 --port=8080** – From the frontend directory, use this command to serve the HTML and JavaScript files. The --host=0.0.0.0 flag makes the site accessible from other devices on the same network for testing purposes.

5.11 Testing the System

After completing the backend, frontend, database, and smart contract configurations, the system is ready for full functionality testing. This stage ensures that each user role (admin and voter) can interact with the platform correctly, and that vote transactions are recorded and reflected on the blockchain. The frontend also visualizes voting results using real-time charts powered by Chart.js.

- **Open the frontend page in a browser** – Access the system via the live server URL (e.g., <http://localhost:8080> or <http://192.168.x.x:8080>) to begin interaction with the voting interface.
- **Log in as admin to manage blocs** – Use pre-configured admin credentials to access the admin panel. From here, you can add or remove university and college voting blocs and view voting results in real time.
- **Log in as a user to cast votes** – Using a regular user account, access the voting interface, select a bloc, and cast a vote. Transactions will be signed through MetaMask and recorded on the blockchain.
- **View real-time blockchain voting results displayed as charts** – After voting, the results chart updates automatically by querying the smart contract. This confirms that the vote was recorded and counted successfully.



ACCOUNTS	BLOCKS	TRANSACTIONS	CONTRACTS	EVENTS	LOGS
CURRENT BLOCK: 10 GAS PRICE: 20000000000 GAS LIMIT: 6721975 HARDFORK: MERGE NETWORK ID: 1337 RPC SERVER: HTTP://0.0.0.0:7545 MINING STATUS: AUTOMINING WORKSPACE: QUICKSTART SAVE SWITCH ⚙️					
HD PATH: m/44'/60'/0'/0'account_index					
mnemonic: noise patrol pill kick recipe unfold devote nest light detect mass cargo					
ADDRESS		BALANCE	TX COUNT	INDEX	
0xEE657e39F373B538EA845447731e5a0E85C96AEf		99.98 ETH	10	0	🔗
ADDRESS		BALANCE	TX COUNT	INDEX	
0xc039eF039510eDcB88574aE50ea9Eb963B29a77a		100.00 ETH	0	1	🔗
ADDRESS		BALANCE	TX COUNT	INDEX	
0x41689bd6BaA68D2dAF1A30F01eb129BE13f9AdA3		100.00 ETH	0	2	🔗
ADDRESS		BALANCE	TX COUNT	INDEX	
0x1037D66F2C582A5E8C0993a67cD1e26Fda8de749		100.00 ETH	0	3	🔗
ADDRESS		BALANCE	TX COUNT	INDEX	
0xC7Ad25e3eEaB7C7E1217Aa755D82e194c690B349		100.00 ETH	0	4	🔗
ADDRESS		BALANCE	TX COUNT	INDEX	
0x0BF6A9240ad43f929B36CCb31834FcE729579e95		100.00 ETH	0	5	🔗
ADDRESS		BALANCE	TX COUNT	INDEX	
0x989d39194db24D4828B661E350465bc63845868e		100.00 ETH	0	6	🔗

Figure 11: Ganache Local Blockchain Environment – Account Overview.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Introduction

This chapter presents the testing process and findings of the developed blockchain-based voting system. The primary objectives were to validate its **functionality**, **security**, **real-time responsiveness**, and **usability**. Unlike traditional centralized voting platforms, this system relies on a decentralized blockchain infrastructure—specifically, Ethereum deployed locally via Ganache—ensuring **transparency**, **immutability**, and **resistance to tampering**. The system was tested with multiple user roles and voting flows, and both frontend and backend components were critically assessed.

6.2 System Testing Overview

6.2.1 Roles Tested

Two distinct roles were tested to simulate realistic usage:

- **Admin:** Accessed the administrative panel, managed university and college blocs, and monitored live voting results.
- **User:** Logged in as a student, selected university and college blocs, submitted votes, and viewed real-time charts.

6.2.2 Bloc Setup

During deployment, the system automatically initialized the following:

- **3 University Blocs**
- **6 College Blocs**, divided into:
 1. 3 engineering blocs
 2. 3 medicine blocs

These blocs were stored in the MySQL database and dynamically filtered on the frontend based on the user's college information retrieved at login.

6.2.3 Voting Results Summary

- **Number of student voters:** 3
- **University Bloc Results:**
 1. Bloc 1: 2 votes
 2. Bloc 2: 1 vote
- **College Bloc Results:**

1. Engineering Bloc 1: 2 votes
 2. Medicine Bloc 2: 1 vote
- **Chart Accuracy:** Vote counts were accurately reflected in the frontend using Chart.js.
 - **Smart Contract Confirmation:** Each vote submission was confirmed through MetaMask and verified on the blockchain via Ganache logs.
 - **Role Isolation:** Admin and user interfaces functioned independently and securely.

6.3 Screenshots and Visual Evidence

Figure 12 displays the login screen for the voting system. It is the first interaction point for both students and admin users. The interface collects user credentials (email and password) and ensures that only registered accounts can proceed. Once validated through backend logic, the user is redirected to the appropriate interface based on their role, which is retrieved from the database and included in the login response.

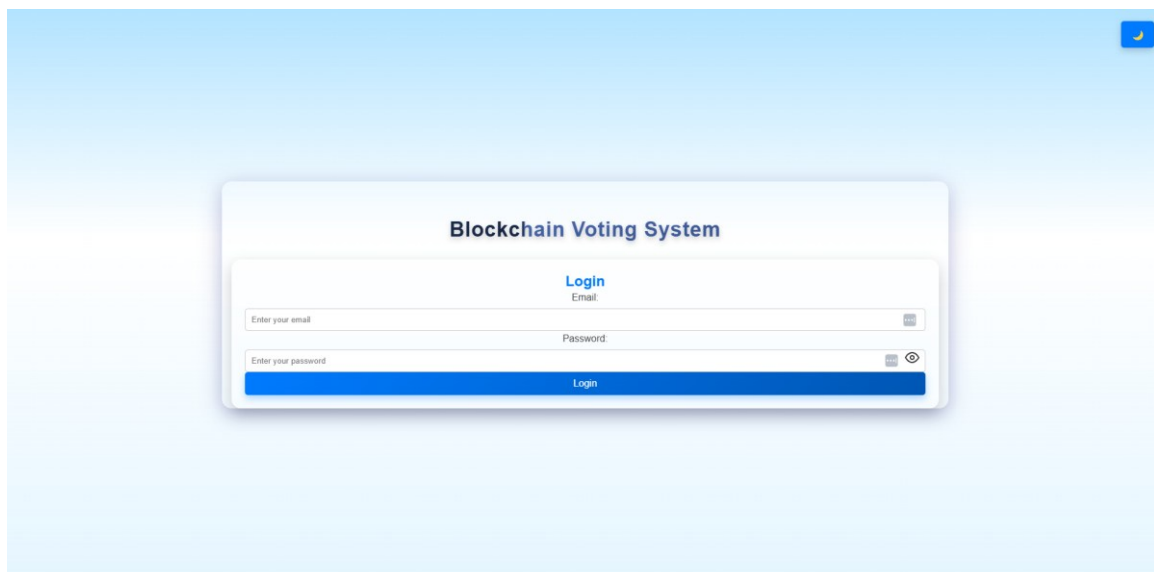


Figure 12: User Login Interface

Figure 13 shows the main voting interface presented to authenticated student users. The system dynamically renders university and college bloc options based on the user's associated college, which is determined during login. The user selects one bloc from each level and submits the vote. Below the form, Chart.js renders real-time vote distributions, allowing voters to see immediate updates following a successful transaction. Users are redirected based on their role.

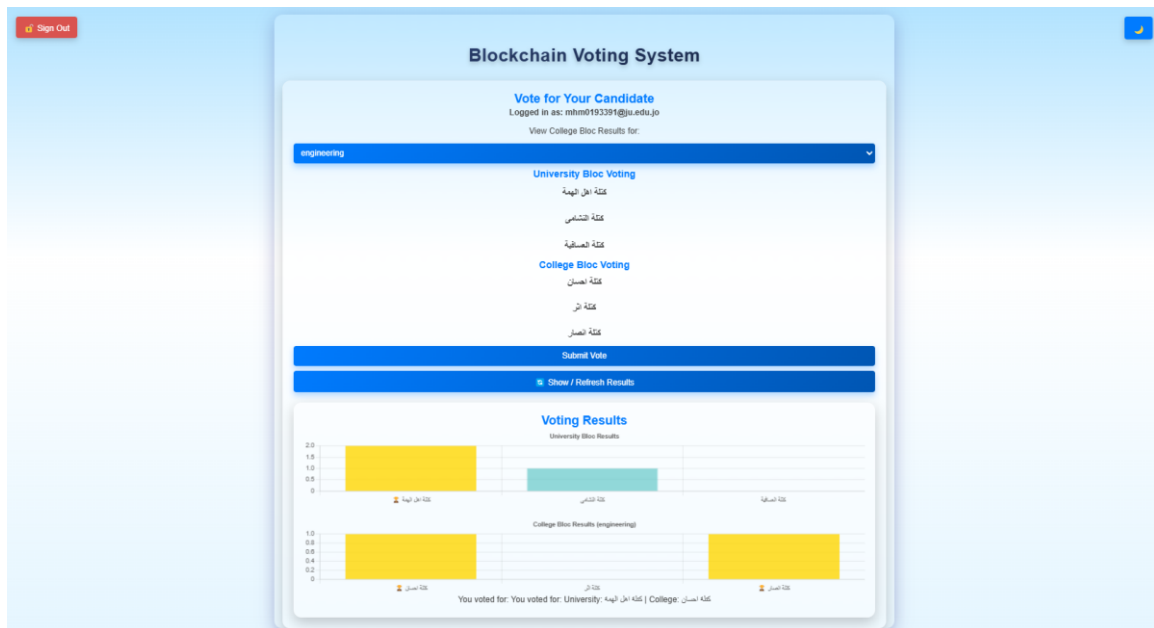


Figure 13: Voting Interface

Figure 14 presents the admin panel, which is accessible only to users with the admin role. The panel enables administrators to manage both university and college blocs. Admins can add new blocs, edit existing ones, or delete them. Real-time voting charts for both levels are displayed using Chart.js, allowing administrators to visually monitor vote progress and distribution without needing to access the blockchain directly.

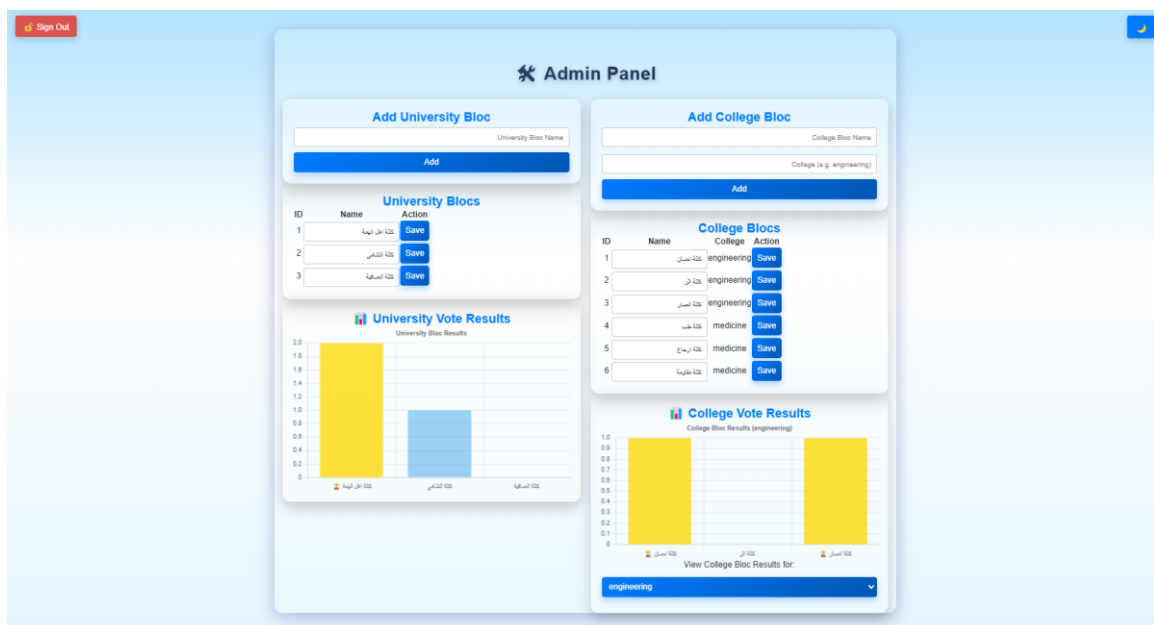


Figure 14: Admin Dashboard

Figure 15 illustrates the MetaMask confirmation dialog that appears when a user submits their vote. This confirmation step ensures that the transaction is authorized by the user and signed

with their private wallet key. The popup includes key transaction metadata such as gas limit, gas fees, sender and receiver addresses, and transaction hash. This acts as both a security layer and a transparency feature

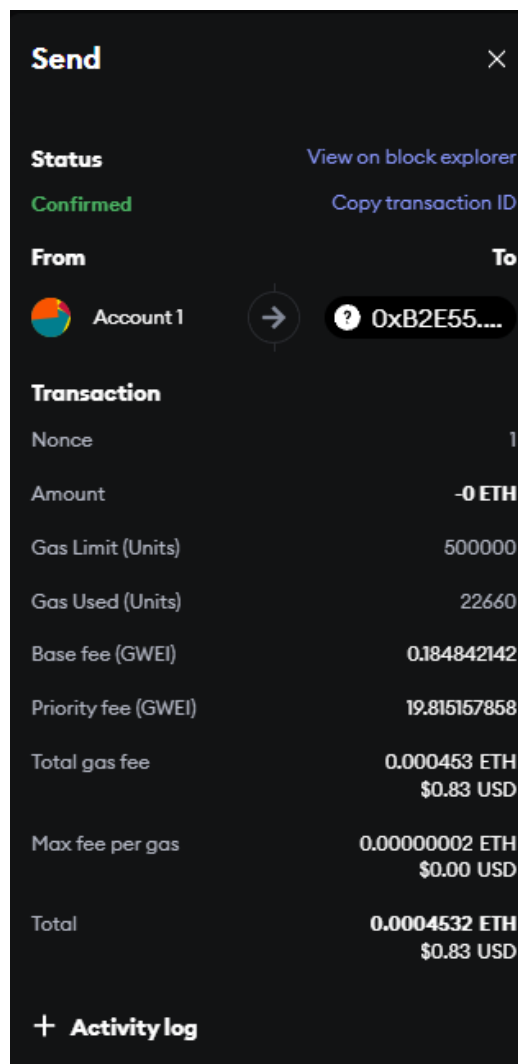


Figure 15: MetaMask Transaction Confirmation

Figure 16 presents the Ganache console log output from a successful vote transaction. It includes a transaction hash, gas usage, and timestamp. This confirms that the vote was successfully mined and included in the blockchain, providing immutable proof of submission.

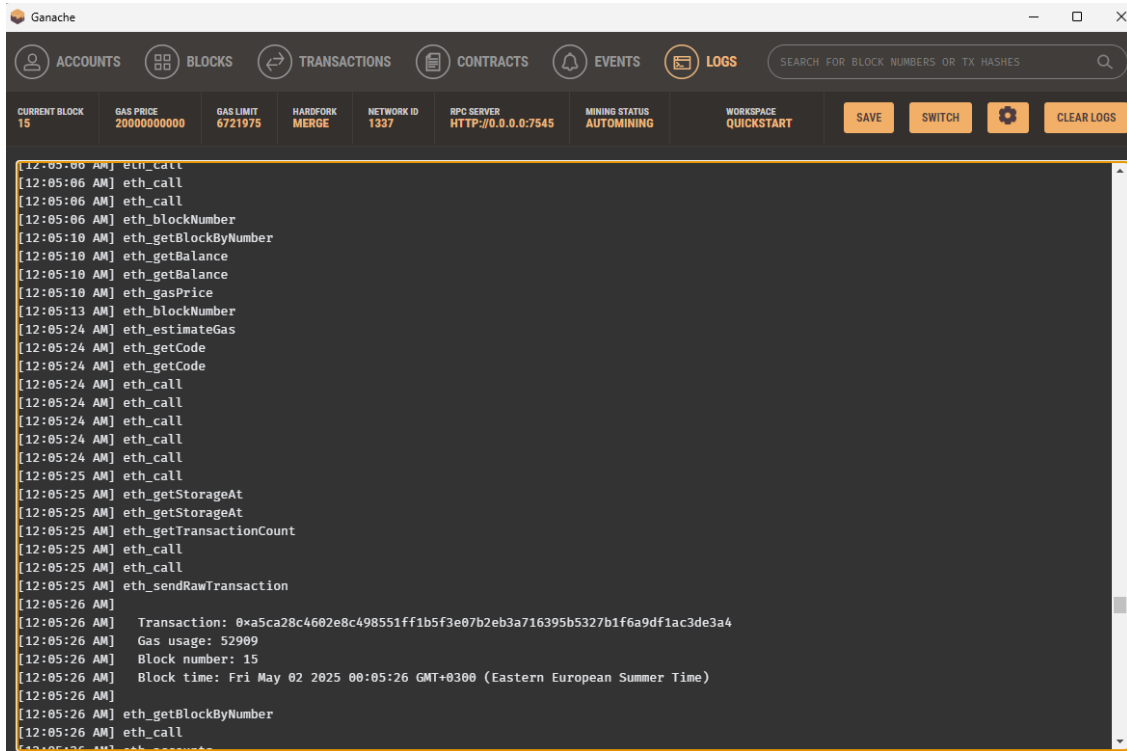


Figure 16: Ganache Successful Transaction Log

Figure 17 shows a failed transaction attempt logged in Ganache. The error message indicates that the user has already voted, triggered by the smart contract's require() condition. This validates the system's enforcement of vote uniqueness by rejecting repeat submissions from the same wallet or hashed email.

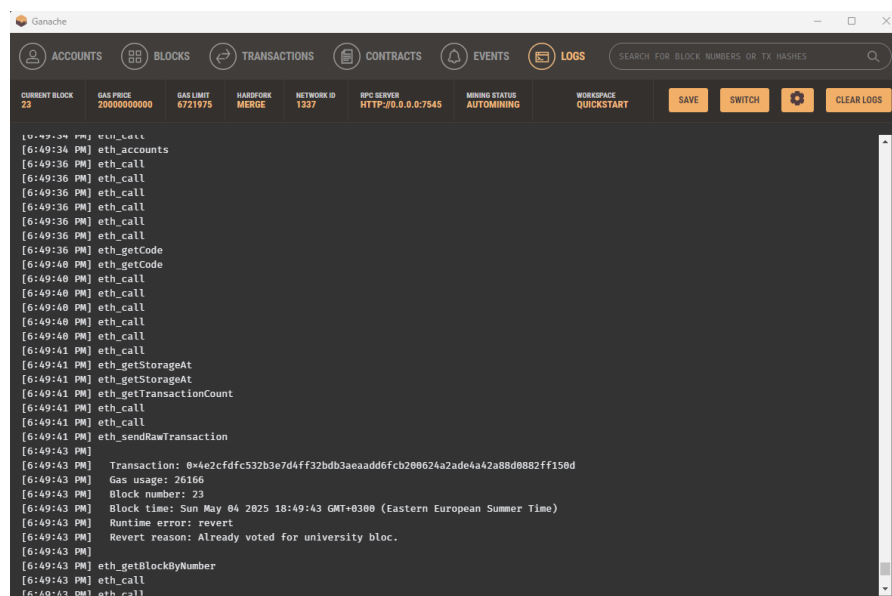
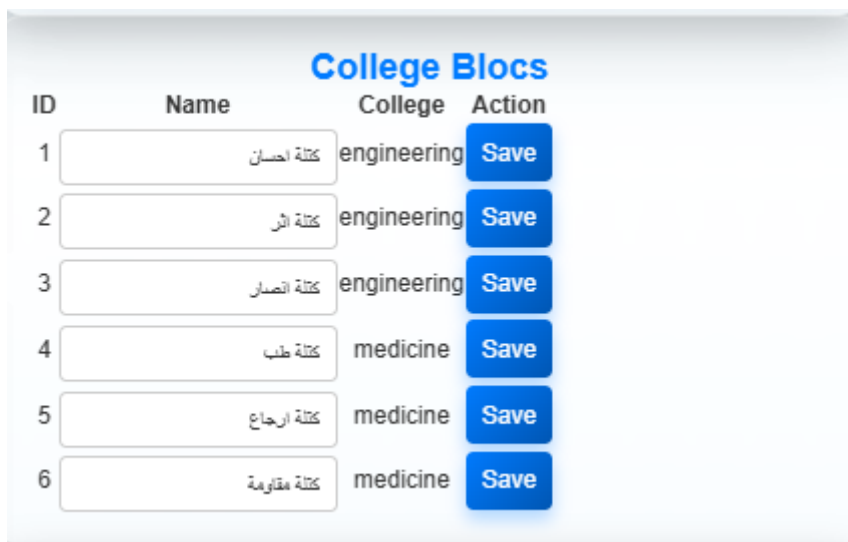


Figure 17: Ganache Failed Transaction Log

Figure 18 captures the backend SQL representation of college blocs. The table lists bloc names, IDs, and associated college fields. It demonstrates the successful storage and retrievability of bloc configurations in the MySQL database. These values are synced with the blockchain and used for rendering options in both user and admin interfaces.



ID	Name	College	Action
1	كتلة احسان	engineering	Save
2	كتلة اثر	engineering	Save
3	كتلة الصار	engineering	Save
4	كتلة طب	medicine	Save
5	كتلة ارجاع	medicine	Save
6	كتلة مقايمة	medicine	Save

Figure 18: College Bloc SQL Table

6.4 Discussion

6.4.1 Achievements

- The implemented blockchain-based voting system achieved the following milestones during development and testing:
- Successfully enabled decentralized voting across two distinct levels: university-wide blocs and college-specific blocs.
- Integrated real-time vote visualization using Chart.js, providing immediate feedback to voters and administrators.
- Ensured secure role-based access separation between students and administrators, with independent user interfaces and data scopes.
- Recorded all votes immutably on the Ethereum blockchain, guaranteeing verifiable and tamper-resistant audit trails.
- Implemented dynamic filtering of college bloc options based on the authenticated user's associated college, ensuring a personalized and context-aware voting experience.

6.4.2 Challenges and Solutions

- Throughout development, several technical and architectural challenges were encountered. Each issue was addressed methodically, resulting in a more stable and secure system:

- **Challenge:** Preventing double voting across multiple devices.
Resolution: The system stores a hashed representation of the user's email on-chain and cross-validates it with the MetaMask wallet address to ensure single-vote enforcement.
- **Challenge:** MetaMask connectivity inconsistencies during development.
Resolution: All services were standardized using the IP address 0.0.0.0, and static local IPs were applied across the backend server, frontend interface, and MetaMask RPC configuration to ensure consistent communication.
- **Challenge:** Frontend failure to detect the deployed smart contract.
Resolution: The ABI (Application Binary Interface) file was moved to a dedicated frontend-accessible folder (frontend/js/abi) to ensure accessibility at runtime.
- **Challenge:** Raw and non-descriptive blockchain error messages shown to users.
Resolution: Try/catch blocks were used to intercept smart contract reverts, and user-friendly messages were displayed on the frontend to improve usability.
- **Challenge:** Poor mobile responsiveness and layout issues, particularly in RTL (Right-to-Left) language displays.
Resolution: CSS styles were restructured using Flexbox, RTL compatibility was implemented, and responsive testing was conducted on multiple screen sizes.
- **Challenge:** Delayed synchronization of Chart.js after vote submission.
Resolution: Chart updates were delayed using setTimeout() to ensure synchronization only occurred after smart contract confirmations were mined.

6.4.3 Lessons Learned

- Developing decentralized systems is technically rewarding but significantly more complex than centralized architecture. Even minor configuration mismatches (such as RPC IP addresses or chain IDs) can lead to silent failures.
- Web3 development demands strict control of ABI versions and consistency in smart contract deployment across environments.
- While local Storage provided useful client-side session persistence, relying on blockchain state for vote tracking proved essential for security and auditability.
- Although Ganache provides a cost-free development environment, real-world deployments must carefully consider gas fees and network conditions.
- Cross-device testing revealed bugs and inconsistencies not apparent during single-device development, highlighting the importance of distributed testing scenarios.

6.4.4 Comparison to Traditional Systems

- Conventional online voting platforms often rely on centralized backend systems and databases, which are vulnerable to manipulation, data loss, and outages. These platforms also require trust in central authority and often lack transparency for third-party auditing.

- In contrast, the proposed blockchain-based voting system introduces several security and transparency advantages:
- All votes are recorded immutably on the Ethereum blockchain, ensuring data cannot be altered or deleted once submitted.
- The use of MetaMask ensures that votes are signed and authorized directly by users, eliminating the need for centralized authentication.
- The system supports external verification of vote counts and participation by analyzing blockchain transactions, thereby enabling independent auditability without centralized trust.
- Vote integrity is enforced at the smart contract level, reducing the risk of unauthorized behavior or technical manipulation.

6.5 Summary

This chapter has demonstrated that the developed blockchain-based voting system is functionally robust, user-centric, and technically well-structured. Through systematic testing, the system was shown to support secure, decentralized voting processes while maintaining real-time performance and administrative flexibility. Challenges such as blockchain synchronization delays, IP address mismatches, MetaMask connectivity issues, and interface responsiveness were identified during development and resolved through targeted technical interventions. The resulting system architecture is not only secure and scalable but also highlights the practical feasibility of applying blockchain technology in academic and civic voting environments, where transparency, immutability, and voter confidence are paramount.

CHAPTER 7

CONCLUSIONS AND FUTURE WORK

7.1 Conclusions

This project successfully designed, developed, and tested a secure decentralized voting system using blockchain technology for the university environment. The implementation aimed to solve one of the most pressing issues in digital elections — ensuring trust without relying on central authorities.

Our main accomplishment lies in the security model achieved by leveraging smart contracts, Ethereum blockchain immutability, and secure backend authentication using bcrypt-hashed passwords. Every vote is:

- Irreversible
- Publicly auditable
- Bound to a unique voter identity, and
- Resistant to tampering, fraud, or duplication.

The final system enabled:

- Role-based login for admin and users
- Real-time voting visualization
- Dynamic filtering of college-specific blocs
- Secure on-chain vote submission through MetaMask
- Proper syncing across Ganache, Web3, Truffle, and frontend UI

Through extensive testing, we validated the system's ability to operate in a real-world university setting. Challenges such as device-level vote duplication, wallet management, contract synchronizing, and responsive design were all resolved during development.

7.2 Limitations

- The current solution is optimized for local use (via Ganache) and not yet ready for deployment on a live public blockchain.
- It lacks multi-function user pathways, such as support for referendum-style voting or organizational elections.

7.3 Future Work

To expand the impact and usability of this system, several improvements can be considered in future versions:

1. Mobile App Integration

Developing a mobile-friendly version or a dedicated mobile app will enhance accessibility, especially in universities where mobile usage is dominant.

2. Zero-Knowledge Proof Voting (ZKP)

Integrating ZK voting protocols would allow users to vote privately while still proving the integrity of their votes — combining privacy with transparency.

3. Multi-Use Election Engine

Transform the voting engine into a modular system where different departments or faculties can configure their own elections for various purposes.

4. Real Blockchain Deployment

Deploying Ethereum test nets like Goerli or Sepolia, or Layer 2 networks like Polygon, would test performance and gas efficiency under real-world blockchain conditions.

5. Voter Registration on-chain

Consider migrating the voter eligibility check and registration process to the blockchain itself using hashed identity verification.

7.4 Final Remarks

This project demonstrated the real-world potential of blockchain in solving critical electoral problems. The final product is functional, secure, and extensible. While it was specifically implemented for the University of Jordan's academic elections, its architecture was designed with adaptability in mind.

With minimal changes to system parameters and configuration, the same platform can be applied to other academic institutions, professional organizations, or civic electoral environments. More importantly, it provides a solid foundation for future enhancements in decentralized civic technology.

REFERENCES

- [1] Krish-Depani, “GitHub - Krish-Depani/Decentralized-Voting-System: A decentralized voting system using Ethereum blockchain for secure and transparent elections, with features like user authentication and real-time result tracking,” *GitHub*, 2023. [Online]. Available: <https://github.com/Krish-Depani/Decentralized-Voting-System> [Accessed: Mar. 10, 2025].
- [2] M. Lamsani, F. Fadli, and S. Jatmiko, “Electronic Voting Using Decentralized System Based on Ethereum Blockchain,” *Jurnal Ilmiah Komputasi*, vol. 19, no. 1, Mar. 2020. doi: <https://doi.org/10.32409/jikstik.19.1.152>
- [3] S. T. Alvi, M. N. Uddin, L. Islam, and S. Ahamed, “DVTChain: A blockchain-based decentralized mechanism to ensure the security of digital voting system,” *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 9, Jul. 2022. doi: <https://doi.org/10.1016/j.jksuci.2022.06.014>.
- [4] S. Gao and Y. Li, “An empirical study on the adoption of blockchain-based games from users’ perspectives,” *The Electronic Library*, vol. 39, no. 4, pp. 596–614, Oct. 2021. doi: <https://doi.org/10.1108/el-01-2021-0009>.
- [5] E. Landerreche and M. Stevens, “On Immutability of Blockchains,” *European Society for Socially Embedded Technologies (EUSSET)*, Apr. 2018. doi: https://doi.org/10.18420/blockchain2018_04.
- [6] K. Proctor, “Decentralization, Immutability, and Integrity: The Role of Blockchain Technology in Enhancing Cybersecurity,” *ResearchGate*, Jan. 28, 2024. [Online]. Available: https://www.researchgate.net/publication/377751048_Decentralization_Immutability_and_Integrity_The_Role_of_Blockchain_Technology_in_Enhancing_Cybersecurity [Accessed: May 14, 2025].
- [7] G. Walker, “Longest Chain | The Blockchain that Nodes Adopt,” *Learnmeabitcoin.com*, Dec. 19, 2024. [Online]. Available: <https://learnmeabitcoin.com/technical/blockchain/longest-chain/> [Accessed: May 14, 2025].
- [8] Gardhouse Financial, “What is Blockchain?” *gardhousefinancial.ca*, [Online]. Available: <https://gardhousefinancial.ca/what-is-block-chain/>. [Accessed: Jun. 13, 2025].

APPENDICES

The following appendices provide supporting material for the implementation and evaluation of the decentralized blockchain-based voting system. These materials include planning artifacts, source code, visual diagrams, and digital files submitted as part of the final documentation.

A. Project Time Chart

This appendix includes the project timeline visualized as a Gantt chart (see figure below). The chart outlines all major phases of the project, starting with foundational learning and ending with the final presentation delivery.

Figure Gantt Chart: Gantt Chart Blockchain Voting System Project

Refer to the file GanttChart.png included in the appendices folder.

B. Source Code

Key components of the project are represented by the following source files:

- VotingSystem.sol: Solidity smart contract defining the voting logic on Ethereum
- script.js and admin.js: Frontend JavaScript logic for user and admin functionality
- backendwauto.js: Backend server code managing API interactions and blockchain syncing
- hashpassword.js: Utility for securely hashing passwords
- style.css: Styling for both the user and admin interfaces
- index.html and admin.html: HTML templates for system interaction
- sqldata.sql: SQL file defining the database schema and initial data

C. Presentation Slides

The slides used during the final project presentation are included in PDF format. The slides are arranged with two slides per page, as per departmental formatting requirements.

D. Submitted Soft Copy Files

The following files are submitted in soft copy format as part of the final documentation package:

- Project Proposal
- Final Project Documentation
- Presentation Slides

All files are organized and submitted digitally according to departmental guidelines.